Data Structure and Programming, Spring 2025
Written Homework #2

Pei-Yuan, Wu peiyuanwu@ntu.edu.tw
Yi- Chou Lin b10901149@ntu.edu.tw
Chiao-Yun Chin b10901154@ntu.edu.tw
Yi-Chen Wang b10901143@ntu.edu.tw

RELEASE DATE: 03/27/2025
DUE DATE: 04/10/2025

Late policy: Score = Score $\times\ 0.7^{\lceil h \rceil/24}$, h = late hours ≤ 48

1. **(15%) Binary Tree**

   For the following questions, construct a binary tree (or binary trees) given preorder or inorder traversals. If multiple trees exist, provide two possible trees.

   (a) Binary Tree
   Preorder: $7, 9, 3, 2, 15, 6, 18$

   (b) Binary Search Tree
   Preorder: $10, 7, 3, 8, 16, 13, 14$

   (c) Binary Tree
   Preorder: $12, 7, 19, 1, 13, 6, 8$
   Inorder: $19, 7, 12, 13, 6, 1, 8$

2. **(18%) Recursion Problem : Recycling Bottles for Free Soda**

   In a small town, a soda company runs a special recycling promotion. You can buy soda bottles with money and after drinking, you can return a certain number of empty bottles to get a free soda.

   Given:

   - Money (M): The amount of money you have.
   - Price (P): The cost of one soda bottle.
   - Bottles (B): The number of empty bottles required to exchange for free soda.

   Goal: Find out the total number of sodas that you can drink, including the ones bought directly and the ones obtained through bottle exchanges.

(a) Please fill in the blanks and complete the following Python code.

(b) Please analyze the time complexity of the algorithm. (As a function of M, P, B. Treat P and B as constants and consider the case where M approaches infinity.) Please, be sure to use all three elements: M, P, and B to represent your answer.

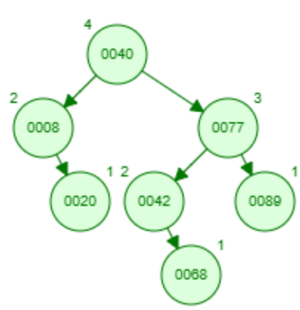
```python
# Function to calculate total sodas
def countSodas(money, price, bottle):
    sodas = money // price          #Θ(1)
    return sodas + countRec(_____,_______)

# Recursive helper function
def countRec(sodas, bottle):
    if sodas < bottle:              #Θ(1)
        return 0                    #Θ(1)
    newSodas = sodas // bottle      #Θ(1)
    return newSodas + countRec(__________________________,
        ______)

# Example usage
money = 15
price = 1
bottle = 3
print(countSodas(money, price, bottle))
```

3. **(21%) AVL Tree**

(a) Show the result after inserting key 62 in the AVL tree shown in the figure below.

(b) Show the result after deleting key 8 in the AVL tree shown in the figure below.

(c) What are all the possible heights for an AVL tree with 493 nodes? Justify you answers. (The height of a leaf in AVL tree is defined as 1.)



4. **(22%)Parentheses Balance Problem**

   (a) Which data structure mentioned in class is used to confirm whether the parentheses are balanced (stack, queue, ordered/unordered linked list, etc.)? Please briefly describe the method and why it works.

   (b) List all possible legal permutations when the set contains three types of brackets: `()`, `[]`, and {}. Each pair of parentheses must be used exactly once. (hint:there are 15 permutations)

   Additionally, the placement rules are as follows:

   - {} can not be inside of `[]`
   - {} and `[]` can not be inside of `()`.
   - `[]`, {} can be empty inside.

   Incorrect examples:

   - `[]()`
   - `[(])`{}
   - `[]()`
   - {}`()`

   Correct examples:

   - {`[]`}`()`
   - {`()`}`[]`
   - `[()]`{}

   (c) Regarding the previous question, suppose that there are the following pairs: () [ ] {}, each having a, b, and c pair, respectively, and all are randomly ordered. How can we determine whether the sequence is legal? Please briefly describe the method and why it works.

5. **(18%)Complexity Analysis**
   Assume T(n) runs in constant time as n is small.

   (a) $T(n) = T(n-2) + n$

   (b) $T(n) = 2T(n-1) + n$

   (c) $T(n) = 2T(n-1) + 3T(n-2) + 1$, $T(1) = T(2) = 1$

   (d) $T(n) = 8T\left(\frac{n}{4}\right) + \log n$

   (e) $T(n) = T\left(\sqrt{n}\right) + 1$

   (f) $T(n) = 3T\left(\sqrt{n}\right) + 1$

6. **(6%) Binary Search Tree**
   Please prove that a binary tree is a binary search tree if and only if its in-order traversal is sorted in ascending order. (Please use the definition to prove it.)